

Git for Developers

Module 2 — Core Git Concepts

Staging, Committing, History, and Undoing Mistakes

Module 2 — Agenda

The Three-Tree Architecture

- Working Directory, Staging Area, and Repository explained
- The Git file lifecycle: untracked → staged → committed

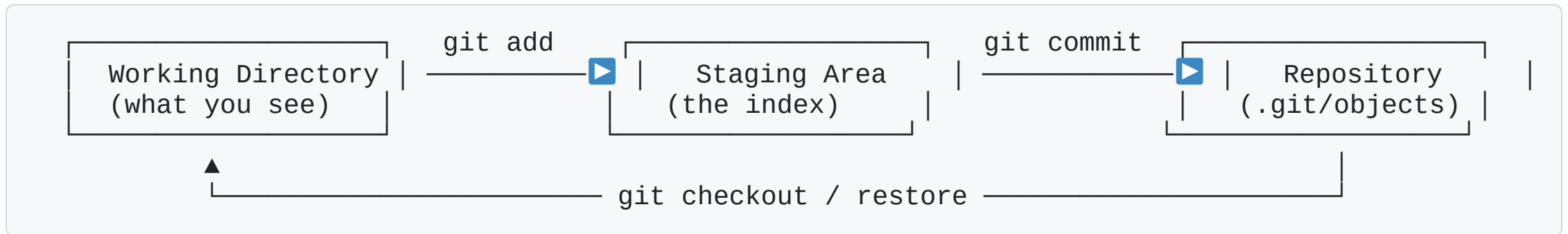
Working With Git

- Creating a repository and making your first commit
- Viewing history with `git log` and `git status`
- Undoing changes: `git restore` , `git reset` , `git revert`

Part 1 — The Three-Tree Architecture

The Three Trees

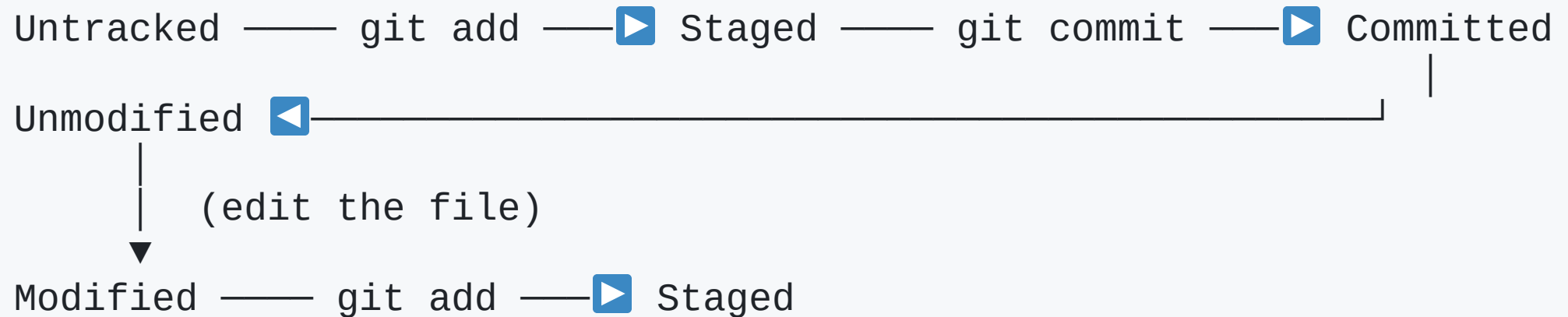
Git manages your project through three distinct areas:



Tree	Also called	Purpose
Working Directory	Working tree	Your files as they exist on disk
Staging Area	Index	Snapshot of what the next commit will contain
Repository	HEAD	All committed snapshots, stored in <code>.git/</code>

The Git File Lifecycle

Every file in your working directory is in one of these states:



State	Meaning
Untracked	New file Git has never seen before
Modified	Tracked file that has been changed since last commit
Staged	File queued for the next commit
Committed	File safely stored in the local repository

Checking Status

`git status` is the most frequently used Git command:

```
$ git status
```

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

```
Changes to be committed:
```

```
(use "git restore --staged <file>..." to unstage)
```

```
    modified:   README.md          ← staged
```

```
Changes not staged for commit:
```

```
(use "git add <file>..." to update what will be committed)
```

```
    modified:   src/app.js          ← modified but not staged
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
    tests/new-test.spec.js         ← untracked
```

Part 2 — Making Your First Commit

Initializing and Staging

```
# Create a new project and initialize Git
mkdir hello-git && cd hello-git
git init

# Create a file
echo "# Hello, Git!" > README.md

# Check status – file is untracked
git status

# Stage the file
git add README.md

# Check status again – file is now staged
git status
```

Staging Strategies

```
# Stage a single file  
git add README.md
```

```
# Stage multiple files  
git add src/app.js src/utils.js
```

```
# Stage all changes in the current directory  
git add .
```

```
# Stage parts of a file interactively (patch mode)  
git add -p README.md
```

```
# Stage all tracked modified files (not new untracked files)  
git add -u
```

Prefer `git add -p` for fine-grained control — it lets you review each change before staging.

Making a Commit

```
# Commit with an inline message
git commit -m "Initial commit: add README"

# Open your configured editor to write the message
git commit

# Stage all modified tracked files and commit in one step
git commit -am "Fix typo in README"
```

What a commit records

- A SHA-1 hash (unique identifier, e.g. `a3f7c21`)
- Your name, email, and timestamp
- The commit message
- A pointer to the project tree snapshot
- A pointer to the parent commit

Writing Good Commit Messages

```
feat: add user authentication endpoint
```

```
Add JWT-based login and logout endpoints.
```

- POST /auth/login returns a signed JWT
- POST /auth/logout invalidates the token
- Includes rate limiting (5 req/min per IP)

```
Closes #42
```

The golden rule

- **Subject line:** 50 characters max, imperative mood ("add", "fix", "update")
- **Blank line** separating subject from body
- **Body:** explain *why*, not *what* (the diff shows what)

Bad: "fixed stuff" | Good: "fix: prevent login bypass when token is expired"

Part 3 — Viewing History

git log — Browsing Commits

Default — full log with author, date, and message

```
git log
```

Compact one-line view

```
git log --oneline
```

Graph view showing branches and merges

```
git log --oneline --graph --all
```

Limit to the last N commits

```
git log -5
```

Search commit messages

```
git log --grep="authentication"
```

Show commits by a specific author

```
git log --author="Jane"
```

Reading `git log` Output

```
$ git log --oneline
a3f7c21 (HEAD -> main, origin/main) feat: add user authentication
b2e1d40 fix: handle null pointer in login flow
c9a8b12 docs: update README with setup instructions
d7f3e56 chore: add .gitignore for Node.js project
e1c2a34 Initial commit: add README
```

Part	Meaning
<code>a3f7c21</code>	Abbreviated SHA hash — unique commit ID
<code>(HEAD -> main)</code>	Your current position and branch
<code>origin/main</code>	Where the remote branch is
Message	What changed in this commit

git show — Inspect a Commit

```
# Show the most recent commit and its diff  
git show
```

```
# Show a specific commit  
git show a3f7c21
```

```
# Show only the files changed in a commit  
git show --stat a3f7c21
```

```
# Show a specific file at a specific commit  
git show a3f7c21:src/app.js
```

`git diff` — What Changed?

```
# Changes in working directory vs. staging area (unstaged changes)  
git diff
```

```
# Changes in staging area vs. last commit (staged changes)  
git diff --staged
```

```
# Compare two commits  
git diff a3f7c21 b2e1d40
```

```
# Compare two branches  
git diff main feature/auth
```

```
# Compact summary of what changed  
git diff --stat main feature/auth
```

Part 4 — Undoing Mistakes

Three Ways to Undo — Overview

Scenario	Command	Safe for shared branches?
Discard unstaged changes	<code>git restore <file></code>	Yes — local only
Unstage a file	<code>git restore --staged <file></code>	Yes — local only
Undo the last commit (keep changes)	<code>git reset --soft HEAD~1</code>	Only if not pushed
Undo the last commit (discard changes)	<code>git reset --hard HEAD~1</code>	Only if not pushed
Undo a pushed commit safely	<code>git revert <hash></code>	Yes — creates new commit

`git restore` — Discard Working Directory Changes

```
# Discard all changes to a file (restore to last commit)
git restore README.md

# Discard all unstaged changes in the entire repo
git restore .

# Unstage a staged file (keep the changes in working directory)
git restore --staged README.md

# Restore a file from a specific commit
git restore --source a3f7c21 README.md
```

`git restore` is safe — it only affects your working directory and staging area, never committed history.

git reset — Move HEAD Backward

```
# Soft reset – undo commit, keep changes staged
git reset --soft HEAD~1

# Mixed reset (default) – undo commit, keep changes unstaged
git reset HEAD~1
git reset --mixed HEAD~1

# Hard reset – undo commit AND discard all changes (destructive!)
git reset --hard HEAD~1

# Reset to a specific commit
git reset --hard a3f7c21
```

Reset modes explained

Mode	Commit	Staging Area	Working Directory
<code>--soft</code>	Undone	Unchanged (staged)	Unchanged
<code>--mixed</code>	Undone	Cleared	Unchanged
<code>--hard</code>	Undone	Cleared	Discarded

Use `--soft` when you want to recommit with a better message. Use `--hard` only locally.

git revert — Safe Undo for Shared Branches

```
# Create a new commit that undoes the changes of a previous commit  
git revert a3f7c21
```

```
# Revert the last commit  
git revert HEAD
```

```
# Revert without immediately committing (stage only)  
git revert --no-commit a3f7c21
```

Why `revert` is safe for shared branches

- It does **not** rewrite history
- It creates a **new commit** that inverts the changes
- Your teammates' history is not affected

Use `git revert` on `main` and shared branches. Use `git reset` only on local, unpublished work.

`git commit --amend` — Fix the Last Commit

```
# Fix the commit message of the last commit
git commit --amend -m "fix: correct typo in authentication header"

# Add a forgotten file to the last commit
git add forgotten-file.js
git commit --amend --no-edit
```

`--amend` rewrites the last commit's SHA. Only use it **before** pushing to a shared remote.

Summary — Core Concepts

Command	What it does
<code>git status</code>	Show working directory and staging area status
<code>git add <file></code>	Stage a file for the next commit
<code>git commit -m "..."</code>	Create a commit with staged changes
<code>git log --oneline</code>	Compact history view
<code>git diff</code>	Show unstaged changes
<code>git diff --staged</code>	Show staged changes
<code>git restore <file></code>	Discard unstaged changes
<code>git reset --soft HEAD~1</code>	Undo last commit, keep changes
<code>git revert <hash></code>	Safely undo a pushed commit